PROJECT GUIDE

VaultSync

A Chrome password manager: encryption, capture, auto-fill and Google Drive sync - explained in simple words.

TypeScript · Manifest V3 · Web Crypto · AES-256-GCM · PBKDF2 · Vite + crxjs · Google Drive API · Playwright

A plain-English explanation of how this password manager works, which technologies it uses and why, plus the key concepts worth knowing. **Made by Ishant · © 2026 IMPS. All rights reserved.** Fingerprint: `IMPS-VS-5FA22D0791658C41`

VERSION
1.0.0

MADE BY
Ishant · © 2026 IMPS

UPDATED
November 2026

Contents

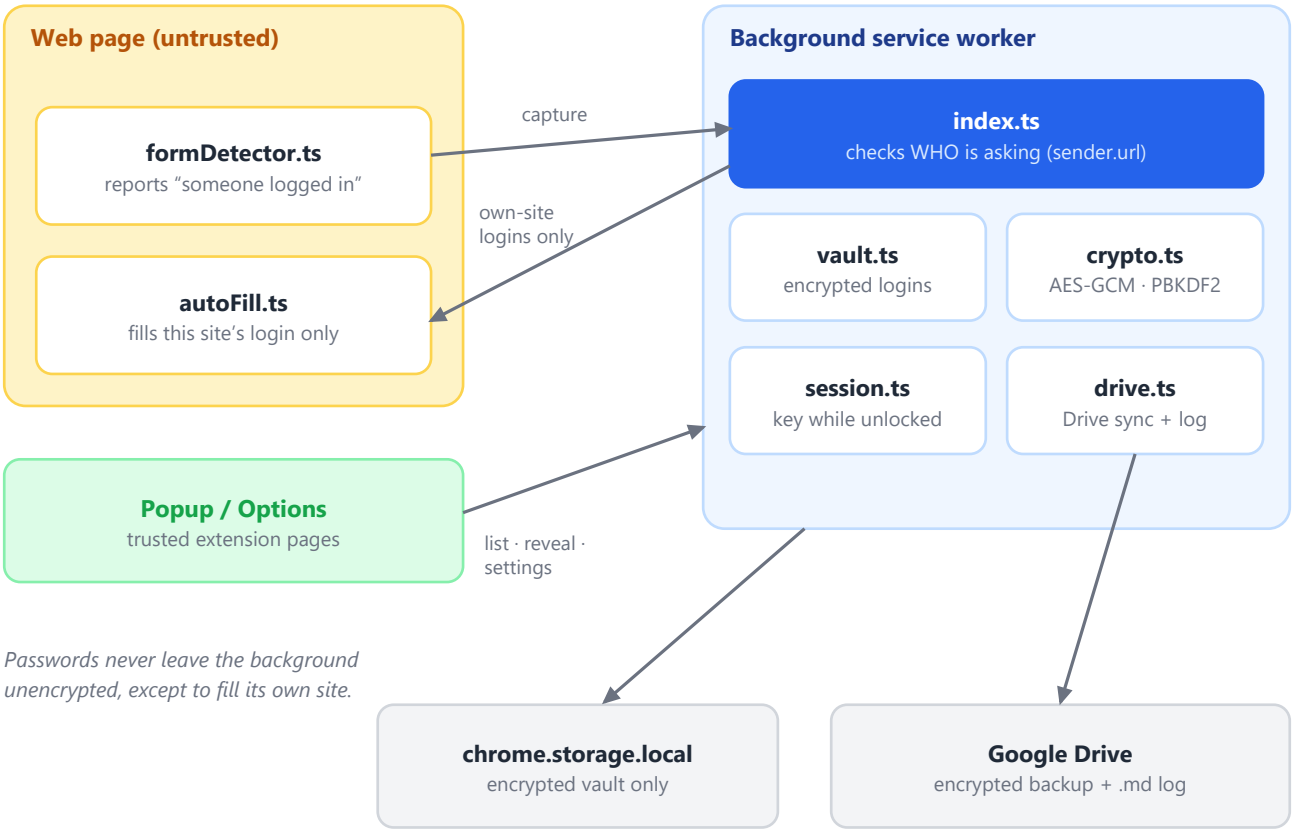
1. What it does (in one minute) . . . 2
2. The big picture . . . 2
3. Technologies and why . . . 2
4. How the encryption works . . . 3
5. Locking and the "sleeping" service worker . . . 3
6. Every feature in simple words . . . 4
7. Key concepts . . . 4
8. Build and test pipeline . . . 5
9. Limitations . . . 5

01 What it does (in one minute)

1. You create a **master password** once.
2. When you log in to a website, VaultSync **notices** and saves the username and password, **encrypted**.
3. Next time you visit that site, it **fills the login in for you**.
4. Optionally, it copies the **encrypted** vault to your **Google Drive** and keeps a readable log of *which* sites you saved (never the passwords).

Without your master password, everything it stores is unreadable gibberish, even to someone who copies your files or Drive.

02 The big picture



Passwords never leave the background unencrypted, except to fill its own site.

Who can talk to whom: the trust boundary inside VaultSync

> **Golden rule:** the web page side can only say "*someone logged in here*" or ask "*do you have a login for here?*". Everything powerful (listing, revealing, exporting) is only allowed from the extension's own pages.

03 Technologies and why

Technology	What it is	Why
Chrome Extension, Manifest V3	Chrome's current extension platform	Required by Chrome today. It runs background logic in a *service worker*.
TypeScript	JavaScript with types	Catches mistakes before running, e.g. it makes it impossible to put a password into a log row.

Technology	What it is	Why
Vite + @crxjs/vite-plugin	Build tool + Chrome-extension plugin	Bundles TypeScript and HTML into a loadable extension and reads `manifest.json` directly.
Web Crypto API	Encryption built into the browser	Fast, audited and native. No third-party crypto code to trust.
Vanilla HTML/CSS	No UI framework	Small, fast popup with nothing extra to load.
Google Drive REST API v3	Drive over plain HTTPS (`fetch`)	No SDK needed.
chrome.identity	Chrome's Google sign-in	Gets a Drive access token from the Google account signed in to Chrome, and refreshes it automatically.
Vitest / Playwright	Unit tests / real-browser tests	Vitest checks the logic quickly. Playwright loads the real extension into Chromium and logs in to test pages.

04 How the encryption works

```
master password ──PBKDF2-SHA256, 600,000 rounds, random salt──► 256-bit key
your logins (JSON) ──AES-256-GCM + random 12-byte IV──────────► encrypted blob
```

- **PBKDF2** turns a password into a key *slowly on purpose*. 600,000 rounds make guessing billions of passwords impractical.
- **Salt** (16 random bytes) makes your key unique even if someone else picks the same password.
- **AES-256-GCM** encrypts the data *and* adds a tamper-proof seal (the auth tag).
- **IV** (12 random bytes, new on every save) is required so the same data never encrypts the same way twice.
- **Wrong password?** The seal check fails, so no password hash ever needs to be stored.

The stored envelope looks like `{ v, kdf: {iterations, salt}, cipher, iv, ct }`, base64-encoded.

05 Locking and the "sleeping" service worker

Chrome **stops an extension's background worker after about 30 seconds idle**. If the key lived only in its memory, the vault would lock every 30 seconds.

VaultSync instead keeps the key in `chrome.storage.session`:

Property	Meaning
RAM only	Never written to disk
Wiped on browser close	Closing Chrome always locks the vault
`TRUSTED_CONTEXTS`	Web-page scripts can't read it

Lock (button) or **auto-lock** (no use for 5/15/30/60 minutes, checked every minute by `chrome.alarms`) deletes it.

06 Every feature in simple words

- **Capture:** listens for form *submit*, clicks on "Log in / Sign in / Next" buttons, and Enter in a password box. It finds the username field next to the password (autocomplete hints, email type, names like "user/email/login", or the nearest box before it).
- **Smart password choice:** on sign-up or change-password forms, it picks the *new* password, not the old one or the confirmation.
- **No duplicates:** the same site and username updates the password, and the old one is kept in **history** (last 5).
- **Auto-fill:** about 1 s after a page loads, it asks for logins *for this exact site* and fills them using the **native value setter** plus `input`/`change` events, so React, Angular and Vue notice.
- **SPAs:** a tiny script in the page's own JavaScript world reports `history.pushState` navigations, so auto-fill runs again without a reload.
- **Multi-step logins:** the email typed on page 1 is remembered for 5 minutes (in RAM) and paired with the password on page 2.
- **iframes:** runs in every frame, and each frame only gets logins for *its own* site.
- **Locked capture:** logins captured while locked wait in RAM (badge shows a count) and are saved when you unlock.
- **Drive backup & log:** every change marks the backup "dirty". A debounced sync (at most one every 10 s, plus every 30 min) uploads the encrypted blob and appends rows to the Markdown log.
- **Export / import:** a JSON file containing the same encrypted blob.

07 Key concepts

Concept	Simple meaning
Content script	Extension code that runs inside web pages, in an *isolated world* separate from the page's own scripts.
Isolated vs MAIN world	Two separate JavaScript environments sharing the same page DOM. Patching `history` in the isolated world doesn't affect the page, hence `spaHooks.ts` runs in MAIN.
Service worker	The extension's background script. It starts on demand and is stopped when idle.
Message passing	`chrome.runtime.sendMessage`, the only bridge between page scripts and the background.
`sender.url`	Set by Chrome, so it can't be faked by a page. Used to decide which site a request is for.
Least privilege	Ask for the smallest permissions possible (`drive.file`, no `tabs`/`scripting`).
Debounce	Group many quick events into one action (many saves lead to one Drive sync).
OAuth token	A temporary key Google gives the extension to access Drive on your behalf.
Honeypot field	A hidden input meant to trick auto-fill. VaultSync fills only visible fields.
Fingerprint	Unique marker (`IMPS-VS-…`) in the code and docs, used to recognise copies.

08 Build and test pipeline

```
src/*.ts ──Vite + crxjs──► dist/  (manifest + service worker + standalone content scripts + pages)
                                │
        npm test ─────────► Vitest: crypto · vault · drive (fake Drive API) · content · url   (45 tests)
   npm run test:e2e ──────► Playwright: real Chromium + dist/ + local test sites            (16 tests)
```

The end-to-end tests cover:

- first-run setup
- capture, auto-fill and update-not-duplicate
- React inputs and SPA navigation
- multi-step logins and iframe isolation
- honeypot fields
- a service-worker restart
- lock, wrong password and locked capture
- the popup detail view and manual add
- export/import, the auto-fill toggle and auto-lock

09 Limitations

- **Drive sync needs Google Chrome.**
- **Two devices don't merge:** the last upload wins.
- **Exact site matching only.**
- **The clipboard isn't auto-cleared.**
- **The Drive activity log is readable text:** site names and usernames, which you can switch off.